Background:

Bituos Tools is a local business specializing in tech. They do multiple kinds of services such as app/web development, computer/mobile repair, solar panels/camara installations and selling computers, printers and other accessories. They have plenty of stablished business arrangements with companies being serviced regularly and also deal with individual walk-in customers.

A good record could contain multiple data points such as individual costumer information, repair history, inventory and sales.

Entity Relationship Model

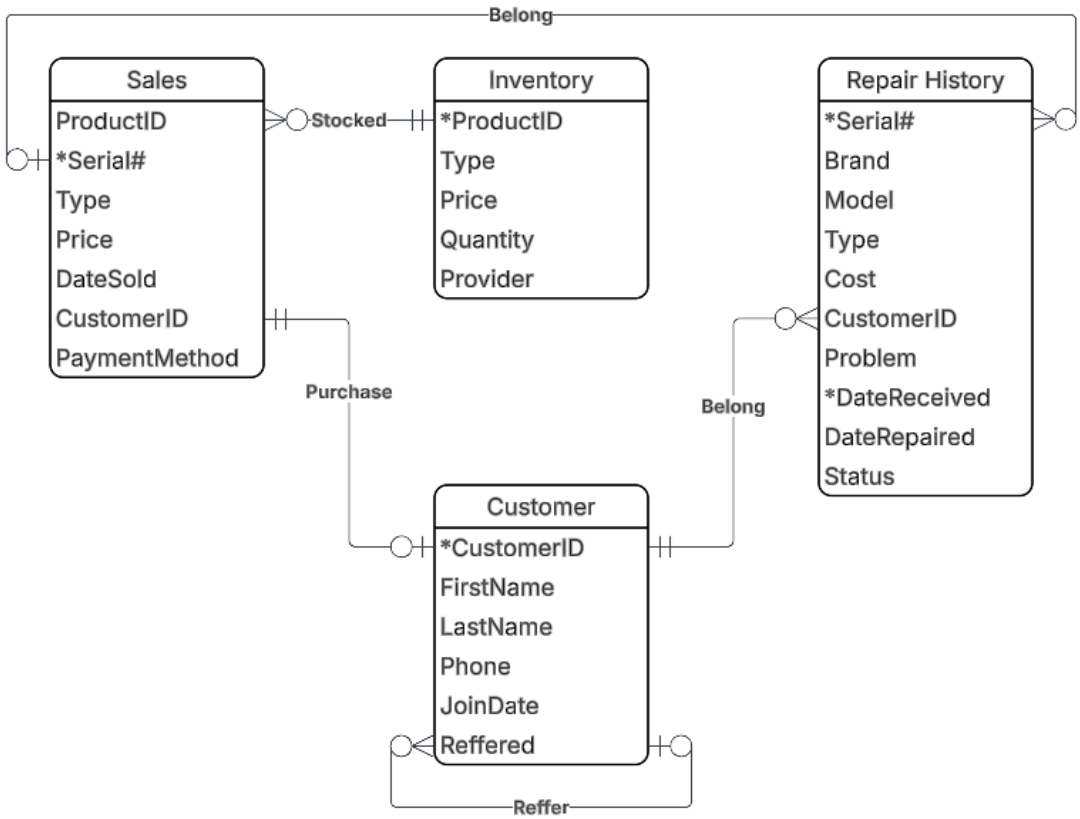
Entities summary:

- Customers: CustomerID, FirstName, LastName, Phone, JoinDate, Referred. CustomerID is the identifier.
- Repair history: Serial#, Brand, Model, Type, Cost, CustomerID, Problem, DateReceived, DateRepaired, Status. Serial# + DateReceived is the identifier.
- Inventory: ProductID, Type, Price, Quantity, Provider. ProductID is the identifier.
- Sales: ProductID, Serial#, Type, Price, DateSold, CustomerID, PaymentMethod. Serial# is the identifier.

Their relationships are:

- Customers may purchase one or more (zero or more) products; A product must be sold to one and only one (exactly one) customer.
- Customers may submit for repair one or more (zero or more) products; A product must be submitted by one and only one (exactly one) customer.
- A product submitted for repair may have been sold by Bituos (zero or one); A product sold by Bituos may be submitted for repair one or more times (zero or more).
- A product line on the inventory may have been sold (zero or more), a product must have been in the inventory to be sold (exactly one).
- A customer may be referred by one (zero or one) customer; and a customer may refer one or more (zero or more) customers.

Entity Relationship Diagram



Relational Model

- Customers (CustomerID, FirstName, LastName, Phone, JoinDate, Referred(fk)).
- Repair history (Serial#, DateReceived, Brand, Model, Type, Cost, CustomerID (fk), Problem, DateRepaired, Status).
- Inventory (ProductID, Type, Price, Quantity, Provider).
- Sales (Serial#, ProductID(fk), Type, Price, DateSold, CustomerID(fk), PaymentMethod).

Functional Dependencies:

- Customers (CustomerID, FirstName, LastName, Phone, JoinDate, Referred(fk)).
    - FD1: CustomerID → FirstName, LastName, Phone, JoinDate, Referred
- Repair history (Serial#, DateReceived, Brand, Model, Type, Cost, CustomerID (fk), Problem, DateRepaired, Status).
    - FD1: Serial#, DateReceived → Brand, Model, Type, Cost, CustomerID (fk), Problem, DateRepaired, Status
    - FD2: Serial# → Brand, Model, Type, CustomerID
    - FD3: Model → Type
- Inventory (ProductID, Type, Price, Quantity, Provider).
    - FD1: ProductID → Type, Price, Provider, Quantity, Provider
- Sales (Serial#, ProductID(fk), Type, Price, DateSold, CustomerID(fk), PaymentMethod).
    - FD1: Serial# → ProductID, Type, Price, DateSold, CustomerID, PaymentMethod
    - FD2: ProductID → Type, Price

Normalization to 3NF

- Customers and Inventory relations are in 3NF, because they are in 1NF; they have no partial functional dependencies so they are in 2NF; and they have no transitive functional dependencies so they are in 3NF.
- Repair history relation is in 1NF. However, it is not in 2NF because FD2: Serial# → Brand, Model, Type, CustomerID. Serial# as part of the primary key, determines non-primary-key attributes. This leads to a partial functional dependency. We need to normalize Repair history to 2NF:
    - Create a new relation to put Serial#, Brand, Model, Type, CustomerID and modify Repair history:
    - Customer item (Serial#, Brand, Model, Type, CustomerID(fk))
        - FD1: Serial# → Brand, Model, Type, CustomerID
        - FD2: Model → Type
    - Repair history (Serial#(fk), DateReceived, Cost, Problem, DateRepaired, Status)
        - FD1: Serial#, DateReceived → Cost, Problem, DateRepaired, Status

- Customer item relation is in 2NF. However, it is not in 3NF because of FD2: Model → Type. Serial# → Model, and Model → Type is a transitive functional dependency. We need to normalize Customer Item to 3NF:
    - Create a new relation to put Model and Type and modify Customer item:
    - Models (Model, Type)
        - FD1: Model → Type
    - Customer item (Serial#, Brand, Model(fk), CustomerID(fk))
        - FD1: Serial# → Brand, Model, CustomerID
- Sales relation is in 1NF, and 2NF. However, it is not in 3NF because of FD2: ProductID → Type, Price. Serial → ProductID, and ProductID → Type, Price is a transitive functional dependency. We need to normalize Sales to 3NF:
    - Since Inventory relation has these attributes, we can simply remove them from Sales, and keep ProductID as a foreign key.
    - Sales (Serial#, ProductID(fk), DateSold, CustomerID(fk), PaymentMethod)
        - FD1: Serial# → ProductID, DateSold, CustomerID, PaymentMethod

Final Output

- Customers (CustomerID, FirstName, LastName, Phone, JoinDate, Referred(fk)).
    - FD1: CustomerID → FirstName, LastName, Phone, JoinDate, Referred
- Inventory (ProductID, Type, Price, Quantity, Provider).
    - FD1: ProductID → Type, Price, Provider, Quantity, Provider
- Repair history (Serial#(fk), DateReceived, Cost, Problem, DateRepaired, Status)
    - FD1: Serial#, DateReceived → Cost, Problem, DateRepaired, Status
- Customer item (Serial#, Brand, Model(fk), CustomerID(fk))
    - FD1: Serial# → Brand, Model, CustomerID
- Sales (Serial#, ProductID(fk), DateSold, CustomerID(fk), PaymentMethod)
    - FD1: Serial# → ProductID, DateSold, CustomerID, PaymentMethod